

Module 2 Lab Exercise: Tools Used in Machine Learning

Group Members

Anashrah Adil

Zain Ahmed

Edwin Alvarado

Kala Hayes

Learning Objectives

By the end of this lab, you will be able to:

- Set up and navigate Jupyter Notebook, Google Colab, and VS Code environments
- Install and import essential Python libraries for machine learning
- Create and format professional documentation using Markdown
- Initialize a GitHub repository for your ML projects
- Understand the basic workflow of data science tools

Prerequisites

- Basic understanding of what machine learning is (Module 1)
- Access to internet for downloading tools and datasets
- A Google account (for Colab) or local Python installation

Part 1: Environment Setup and Tool Overview

What are the main tools we'll use in this course?

Jupyter Notebook/Google Colab: Interactive computing environments where you can write code, see results immediately, and document your work with text and visualizations.

Python Libraries: Pre-written code packages that make machine learning tasks easier:

- **Pandas:** For working with data (like Excel, but more powerful)
- **NumPy:** For mathematical operations on arrays of numbers
- **Matplotlib:** For creating charts and graphs
- **Scikit-learn:** The main library for machine learning algorithms

GitHub: A platform to store, share, and collaborate on code projects

VS Code: A powerful text editor for writing and debugging code

Let's start by setting up our environment!

Environment Setup Instructions

Option 1: Google Colab (Recommended for Beginners)

1. Go to colab.research.google.com
2. Sign in with your Google account
3. Click "New Notebook"
4. You're ready to go! Libraries are pre-installed.

Option 2: Local Jupyter Notebook

1. Install Python from python.org
2. Open terminal/command prompt
3. Run: `pip install jupyter pandas numpy matplotlib scikit-learn`
4. Run: `jupyter notebook`

5. Create a new notebook

Option 3: VS Code

1. Download VS Code from code.visualstudio.com
2. Install Python extension
3. Install Jupyter extension
4. Create a new .ipynb file

For this lab, we recommend starting with Google Colab as it requires no installation.

```
1 # Install required libraries (uncomment if needed)
2 # !pip install pandas numpy matplotlib scikit-learn
3
4 # Import libraries with standard aliases
5 import pandas as pd
6 import numpy as np
7 import matplotlib.pyplot as plt
8 from sklearn import datasets
9 import warnings
10 warnings.filterwarnings('ignore') # Hide warning messages for cleaner output
11
12 print("✅ All libraries imported successfully!")
13 print(f"Pandas version: {pd.__version__}")
14 print(f"NumPy version: {np.__version__}")
```

✅ All libraries imported successfully!
Pandas version: 2.2.2
NumPy version: 2.0.2

Part 2: Loading and Exploring Your First Dataset

We'll use the famous Iris dataset - a classic dataset for beginners. It contains measurements of iris flowers from three different species.

```
1 # Load a simple dataset (Iris flowers - a classic beginner dataset)
2 from sklearn.datasets import load_iris
3
4 # Load the data
5 iris = load_iris()
6 print("Dataset loaded successfully!")
7 print(f"Dataset shape: {iris.data.shape}")
8 print(f"Features: {iris.feature_names}")
9 print(f"Target classes: {iris.target_names}")
```

Dataset loaded successfully!
Dataset shape: (150, 4)
Features: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Target classes: ['setosa' 'versicolor' 'virginica']

```
1 # Convert to pandas DataFrame for easier handling
2 df = pd.DataFrame(iris.data, columns=iris.feature_names)
3 df['species'] = iris.target_names[iris.target]
4
5 # Display first few rows
6 print("First 5 rows of our dataset:")
7 print(df.head())
8
9 print("\nDataset info:")
10 print(df.info())
```

First 5 rows of our dataset:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

species

0	setosa
1	setosa
2	setosa
3	setosa
4	setosa

```

Dataset info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   sepal length (cm)    150 non-null    float64
1   sepal width (cm)     150 non-null    float64
2   petal length (cm)    150 non-null    float64
3   petal width (cm)     150 non-null    float64
4   species              150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None

```

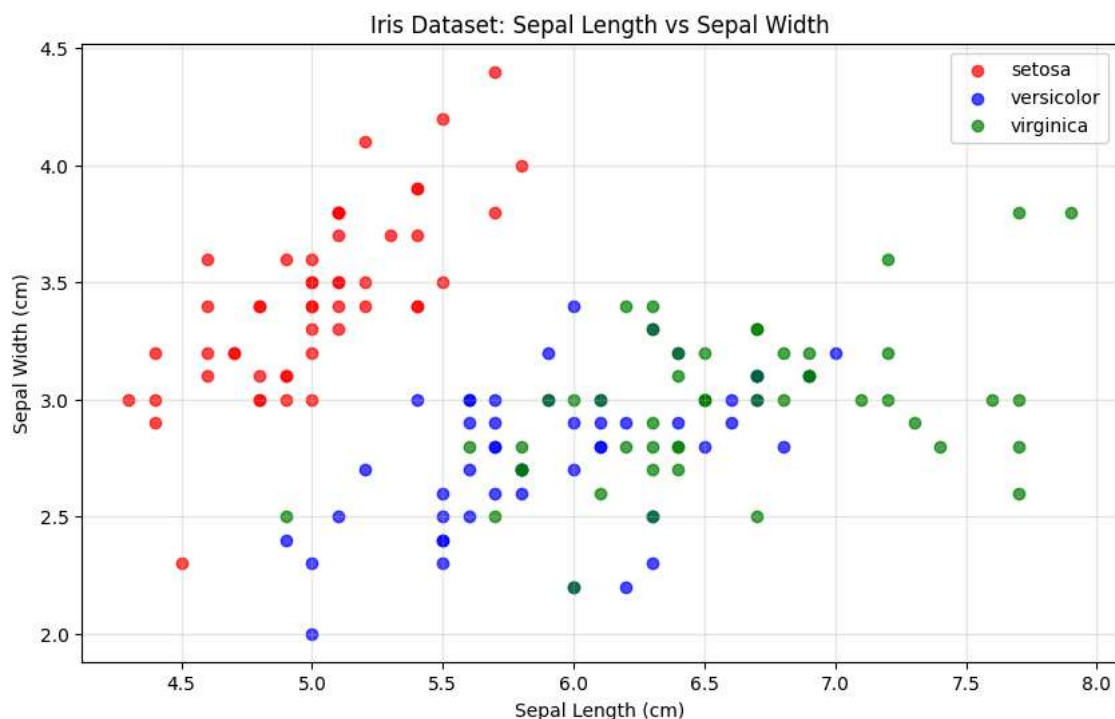
Part 3: Creating Your First Visualization

Data visualization is crucial in machine learning. Let's create a simple plot to understand our data.

```

1 # Create a simple scatter plot
2 plt.figure(figsize=(10, 6))
3
4 # Plot sepal length vs sepal width, colored by species
5 species_colors = {'setosa': 'red', 'versicolor': 'blue', 'virginica': 'green'}
6
7 for species in df['species'].unique():
8     species_data = df[df['species'] == species]
9     plt.scatter(species_data['sepal length (cm)'],
10                species_data['sepal width (cm)'],
11                c=species_colors[species],
12                label=species,
13                alpha=0.7)
14
15 plt.xlabel('Sepal Length (cm)')
16 plt.ylabel('Sepal Width (cm)')
17 plt.title('Iris Dataset: Sepal Length vs Sepal Width')
18 plt.legend()
19 plt.grid(True, alpha=0.3)
20 plt.show()
21
22 print("🎉 Congratulations! You've created your first data visualization!")

```



🎉 Congratulations! You've created your first data visualization!

Part 4: Practice with Basic Data Operations

Let's practice some basic data analysis operations that you'll use throughout the course.

```
1 # Basic statistical analysis
2 print("Basic Statistics for Iris Dataset:")
3 print("=" * 40)
4
5 # Calculate mean values for each species
6 species_means = df.groupby('species').mean()
7 print("\nMean values by species:")
8 print(species_means)
9
10 # Count samples per species
11 species_counts = df['species'].value_counts()
12 print("\nSamples per species:")
13 print(species_counts)
```

Basic Statistics for Iris Dataset:

=====

Mean values by species:

	sepal length (cm)	sepal width (cm)	petal length (cm) \
species			
setosa	5.006	3.428	1.462
versicolor	5.936	2.770	4.260
virginica	6.588	2.974	5.552

	petal width (cm)
species	
setosa	0.246
versicolor	1.326
virginica	2.026

Samples per species:

species	
setosa	50
versicolor	50
virginica	50

Name: count, dtype: int64

Part 5: GitHub and Documentation Best Practices

Why GitHub for Machine Learning?

- **Version Control:** Track changes to your code and data
- **Collaboration:** Work with others on projects
- **Portfolio:** Showcase your work to potential employers
- **Backup:** Never lose your work

Basic GitHub Workflow:

1. **Create Repository:** A folder for your project
2. **Clone/Download:** Get the project on your computer
3. **Add Files:** Put your notebooks and data
4. **Commit:** Save a snapshot of your changes
5. **Push:** Upload changes to GitHub

For This Course:

- Create a repository named "ITAI-1371-ML-Labs"
- Upload each lab notebook as you complete it
- Include a README.md file describing your projects

Action Item: After this lab, create your GitHub account and repository.

✓ Assessment: Tool Familiarity Check

Complete the following tasks to demonstrate your understanding of the tools:

```

1 # Task 1: Create a simple calculation using NumPy
2 # Calculate the mean and standard deviation of sepal length
3
4 sepal_lengths = df['sepal length (cm)']
5
6 # Your code here:
7 mean_sepal_length = np.mean(sepal_lengths)
8 std_sepal_length = np.std(sepal_lengths)
9
10 print(f"Mean sepal length: {mean_sepal_length:.2f} cm")
11 print(f"Standard deviation: {std_sepal_length:.2f} cm")
12
13 # Verification (don't modify)
14 assert isinstance(mean_sepal_length, (float, np.floating)), "Mean should be a number"
15 assert isinstance(std_sepal_length, (float, np.floating)), "Std should be a number"
16 print("✅ Task 1 completed successfully!")

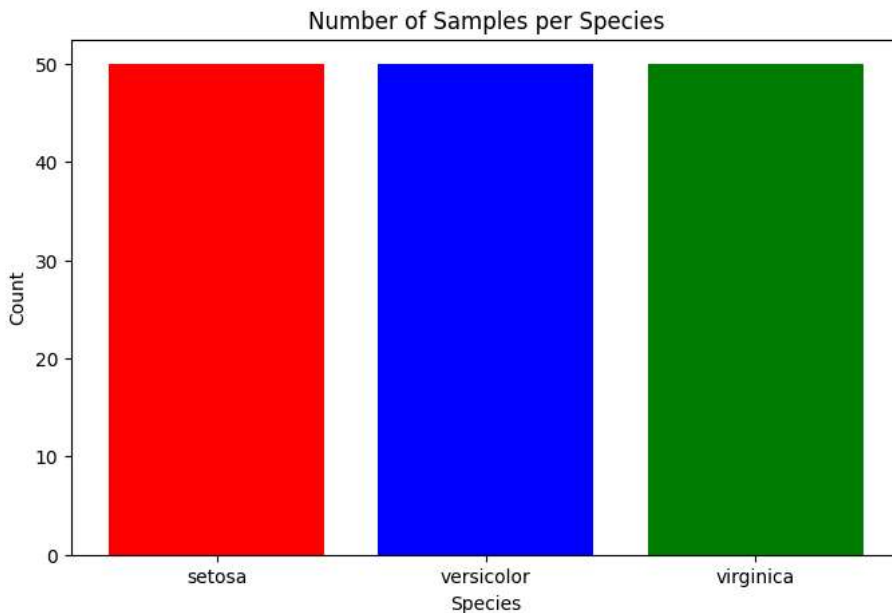
```

Mean sepal length: 5.84 cm
 Standard deviation: 0.83 cm
 ✅ Task 1 completed successfully!

```

1 # Task 2: Create a simple bar chart showing species counts
2 species_counts = df['species'].value_counts()
3
4 plt.figure(figsize=(8, 5))
5 plt.bar(species_counts.index, species_counts.values, color=['red', 'blue', 'green'])
6 plt.title('Number of Samples per Species')
7 plt.xlabel('Species')
8 plt.ylabel('Count')
9 plt.show()
10
11 print(f"Species distribution: {dict(species_counts)}")
12 print("✅ Task 2 completed successfully!")

```



Species distribution: {'setosa': np.int64(50), 'versicolor': np.int64(50), 'virginica': np.int64(50)}
 ✅ Task 2 completed successfully!

Your Analysis and Reflection

Instructions: Complete the analysis below by editing this markdown cell.

My Observations About the Iris Dataset

Dataset Overview:

- Number of samples: [50]
- Number of features: [3]
- Number of classes: [3]

Key Findings from the Visualization:

1. [Write your observation about the scatter plot]
 - In the scatter plot, setosa shows the largest average sepal width among the three species. This indicates that sepal width may help distinguish setosa from the other species.
2. [Write another observation]
 - In the scatter plot, Virginica has the largest average sepal length compared to the other species. This suggests sepal length could be a useful feature for separating Virginica from other species
3. [Write a third observation]
 - According to the scatter plot Versicolor and Virginica appear closer together in both sepal length and sepal width. This overlap suggests these two classes may be more difficult to separate using only these features

Questions for Further Investigation:

- Would creating a scatterplot for petal length and width improve model accuracy? This could test whether more features reduce class overlap.
- [Write another question] How well can this classification model separate the species using only sepal length and sepal width correctly utilizing reinforcement learning with a new dataset?

Reflection: *In 2-3 sentences, describe what you learned about using these tools.*

- Working with these tools showed me that each library has a distinct role yet they combine into one smooth workflow. Pandas organized the Iris data into a readable DataFrame, NumPy handle the calculations like mean and standard deviation, and Matplotlib turned those numbers into visuals that revealed patterns I could not see from the raw data alone. I learned that knowing which tool to reach for at each step is just as important as the analysis itself.

Note: This is practice for documenting your machine learning projects professionally.

✎ PERSONAL EXPERIMENT:

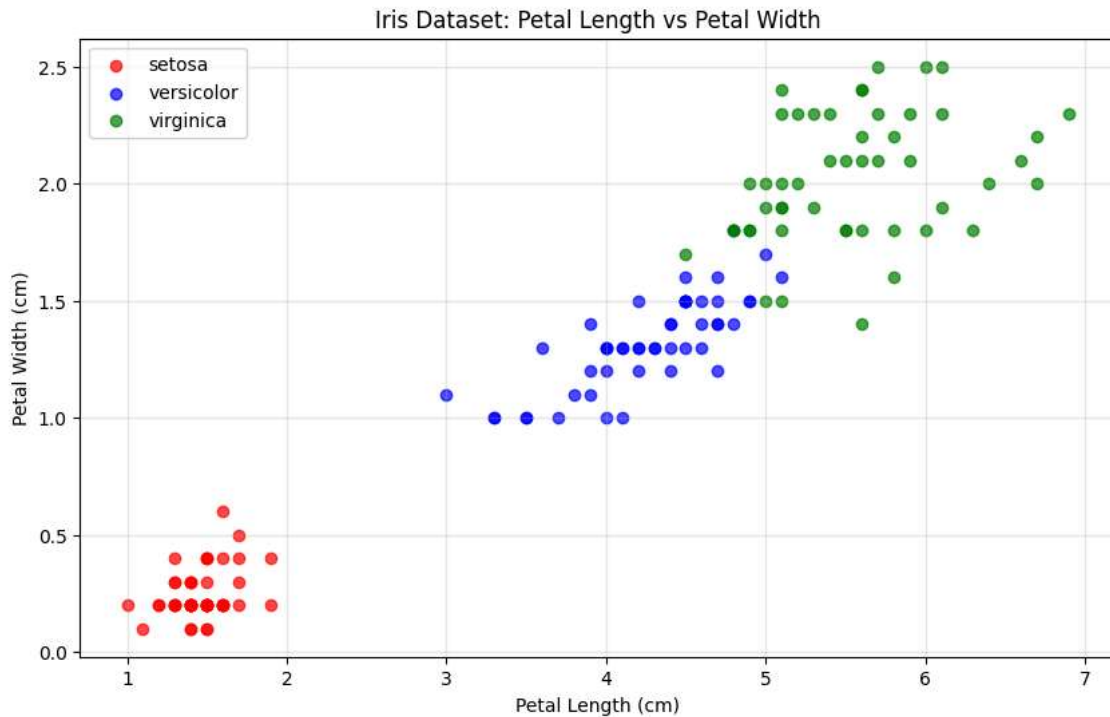
Questions for Further Investigation:

Would creating a scatterplot for petal length and width improve model accuracy? This could test whether more features reduce class overlap.

Following code snippet clarifies the difference between the accuracy level of scatterplot for sepal length and width and scatterplot for petal length and width.

```

1 # Create a scatter plot for Petal Length vs Petal Width
2 plt.figure(figsize=(10, 6))
3
4 species_colors = {'setosa': 'red', 'versicolor': 'blue', 'virginica': 'green'}
5
6 for species in df['species'].unique():
7     species_data = df[df['species'] == species]
8     plt.scatter(species_data['petal length (cm)'],
9                 species_data['petal width (cm)'],
10                 c=species_colors[species],
11                 label=species,
12                 alpha=0.7)
13
14 plt.xlabel('Petal Length (cm)')
15 plt.ylabel('Petal Width (cm)')
16 plt.title('Iris Dataset: Petal Length vs Petal Width')
17 plt.legend()
18 plt.grid(True, alpha=0.3)
19 plt.show()
  
```



ANALYSIS:

In the Iris dataset, petal measurements are more distinct across species than sepal measurements.

Setosa: has tiny and uniform petals.

Versicolor: has medium sized petals.

Virginica: has large, robust petals.

Using sepal features to classify the samples causes huge overlap between versicolor and virginica which can lead to misclassification. Contrary to that Petal Features reduces this overlap and increases model accuracy.

✓ Lab Summary and Next Steps

What You've Accomplished:

- ✓ Set up your machine learning development environment
- ✓ Imported and used essential Python libraries
- ✓ Loaded and explored your first dataset
- ✓ Created your first data visualization
- ✓ Practiced professional documentation with Markdown
- ✓ Learned about GitHub for project management

Preparation for Module 3:

In the next lab, you'll:

- Learn about different types of machine learning
- Build your first simple classifier
- Understand the complete ML workflow
- Work with more complex datasets

Action Items:

1. **Create your GitHub account** and repository
2. **Upload this completed notebook** to your repository
3. **Experiment** with different visualizations using the Iris dataset
4. **Practice** Markdown formatting in a new notebook

Resources for Continued Learning:

- [Pandas Documentation](#)
- [Matplotlib Gallery](#)
- [GitHub Guides](#)
- [Jupyter Notebook Tips](#)

Great job completing Module 2! You're now equipped with the essential tools for machine learning. 🎉

1 Start coding or [generate](#) with AI.